

Apache Kafka: A Framework *for* Handling Real-Time Data Feeds

Apache Kafka is a distributed streaming platform. It is incredibly fast, which is why thousands of companies like Twitter, LinkedIn, Oracle, Mozilla and Netflix use it in production environments. It is horizontally scalable and fault tolerant. This article looks at its architecture and characteristics.



Apache Kafka is a powerful asynchronous messaging technology originally developed by the LinkedIn Corporation, which open sourced it in 2011. It eventually became a part of the Apache project. Kafka, which has been named after the acclaimed German writer, Franz Kafka, is basically a publish-subscribe based fault tolerant messaging system. It can be used to feed events to complex event streaming systems and IoT style automation systems, and also be used with in-memory micro services to provide durability.

In this article, I will explore the architecture and characteristics of Apache Kafka, as well as explain how to install it. This should help readers understand why Kafka is better than a traditional message server.

Kafka often gets used in real-time streaming data architectures to provide real-time analytics. Since it is fast, stable, scalable, reliable and has higher throughput, it is used in cases where open source message brokers like JMS, RabbitMQ and AMQP may not even be considered due to reasons linked to volume and responsiveness. The main reason for Kafka's popularity is its exceptional performance and operational simplicity. In Kafka, the communication between the clients and the servers is done by a simple, high-performance, language-agnostic TCP protocol. This protocol is versioned and maintains backward compatibility with older versions.

Kafka is a robust queue that can handle a high volume of data and enables you to pass messages from one end-point to

another, which is suitable for both offline and online message consumption. Built on top of the Zookeeper synchronisation service, it integrates very well with Apache Storm and Spark for real-time streaming data analysis.

Apache Kafka – the fundamentals

To understand how Kafka works, let's dive in and explore its capabilities from the bottom up. Before going forward, you need to be aware of the main terminology used, such as topics, brokers, producers and consumers.

1. **Topics:** A stream of messages belonging to a particular category or feed name is called a topic, which is a unique term for a Kafka stream. Producer applications write data to topics and consumer applications read from topics.
2. **Partitions:** Each partition is an ordered, immutable sequence of records, identified by its unique sequential ID number called an offset, which is continually appended to a structured commit log.
3. **Brokers:** A broker is a Kafka server. As the name suggests, the producer and consumer don't interact directly but use the Kafka server as an agent or broker to exchange message services. Unlike other message systems, Kafka brokers are stateless.
4. **Clusters:** Since Kafka is a distributed system, it is simply a group of computers, each executing an instance of the Kafka broker.